

Tarea - TIA-06 (Unidad 3 - Tarea 2)

Manipular bases de datos utilizando SGBD para la gestión de la información almacenada.

- **Tarea en Equipo**
- **Peso: 20%**
- **Práctica. Caso de Estudio**
- **DML - Manipulación de Datos**
- **Proyecto de Aula**

MIEMBROS DEL EQUIPO:

- Líder: Franklin Alejandro Torres Florez
- Integrante: Ader Eliecer Quintana Herrera
- Integrante: Samuel Brand Gaviria

Descripción de la Tarea

1. Contextualización

En el desarrollo del proyecto del sistema apícola, se ha identificado que uno de los principales riesgos en la construcción de sistemas de información no radica únicamente en la implementación técnica, sino en errores arrastrados desde las etapas iniciales del modelado conceptual y lógico. En este mismo orden de ideas, esos errores se pueden evidenciar en el modelo físico y afectar la manipulación de los datos y la construcción de consultas:

- Una base de datos física mal relacionada (referencias entre tablas)
- Sin controles para evitar inconsistencias y embasuramiento de datos

Estos problemas impactan directamente la calidad de la información, generando bases de datos:

- Vulnerables
- Inconsistentes y difíciles de mantener
- Ineficientes y de baja calidad
- Propensas a errores en consultas

Por esta razón, antes de iniciar la implementación física, es obligatorio realizar un proceso de: Revisión, corrección y mejora del modelo físico. Este proceso debe garantizar que la base de datos física sea:

- Coherente y correcta estructuralmente
- Protegida para evitar inconsistencias durante procesos de actualización
- Amigable y flexible para realizar consultas con información de calidad

En este contexto, el reto consiste en tomar el modelo físico previamente construido para el sistema apícola y transformarlo en un modelo completamente funcional, implementado en un SGBD, para poder realizar operaciones de inserción, modificación, eliminación y consultas a través del lenguaje de manipulación de datos (DML).

Es importante que al final se realicen pruebas para verificar el cumplimiento de las propiedades ACID (Véase Anexo A)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Informe con resultados

Equipo “02”

(TIA-6: Unidad 3 - Tarea 2)

1.- Inventario final de tablas del sistema de información

Cuadro. Inventario de Tablas definitivas de la Base de Datos

Nro	Tabla	Descripción	Tipo	Tablas Relacionadas
1	usuario	Almacena la información de los usuarios del sistema	E	2, 9, 10
2	rol	Define los roles asignados a los usuarios	E	1
3	apiario	Registra los apiarios gestionados en el sistema	E	4, 5, 11
4	colmena	Contiene la información de las colmenas registradas	E	3, 5, 12
5	sensor	Registra los sensores instalados en las colmenas	E	3, 4, 6, 13
6	lectura_sensor	Almacena las mediciones capturadas por los sensores	E	5
7	producto	Contiene los productos apícolas disponibles para venta	E	8, 14
8	detalle_pedido	Relaciona los pedidos con los productos comprados	R	7, 9
9	pedido	Registra las compras realizadas por los usuarios	E	1, 8
10	usuario_rol	Relaciona usuarios con roles del sistema	R	1, 2
11	apicultor	Registra los datos de los apicultores responsables	E	3
12	estado_colmena	Define los estados posibles de una colmena	E	4
13	tipo_sensor	Clasifica los sensores según su función	E	5
14	categoria_producto	Clasifica los productos del sistema	E	7
15	ubicacion	Almacena la ubicación geográfica de los apiarios	E	3
16	usuario	Almacena la información de los usuarios del sistema	E	2, 9, 10
17	rol	Define los roles asignados a los usuarios	E	1
18	apiario	Registra los apiarios gestionados en el sistema	E	4, 5, 11
19	colmena	Contiene la información de las colmenas registradas	E	3, 5, 12

2.- Script de “*creación*” de las tablas definitivas Bases de Datos (**CREATE**)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

The screenshot shows a PostgreSQL client window with the title 'red_apicola/postgres@PostgreSQL 18'. The interface includes a toolbar with icons for file operations, query execution, and other database functions. The 'Query' tab is active, displaying a SQL script for creating two tables: 'productos_lotes' and 'lotes_produccion'. The script includes primary and foreign key constraints. A tooltip 'Execute script (F5)' is visible over the execution button. The 'Messages' tab is also active, showing the message 'Query returned successfully in 178 msec.' A green notification bar at the bottom right confirms the successful execution. The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.178' and 'CRLF Ln 137, Col 1'.

```
123 id_apiaro INT NOT NULL,
124 id_certificacion INT NOT NULL,
125 PRIMARY KEY (id_apiaro, id_certificacion),
126 FOREIGN KEY (id_apiaro) REFERENCES apiarios(id_apiaro),
127 FOREIGN KEY (id_certificacion) REFERENCES certificaciones(id_certificacion)
128 );
129
130 CREATE TABLE productos_lotes (
131     id_producto INT NOT NULL,
132     id_lote INT NOT NULL,
133     PRIMARY KEY (id_producto, id_lote),
134     FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
135     FOREIGN KEY (id_lote) REFERENCES lotes_produccion(id_lote)
136 );
137
```

CREATE TABLE

Query returned successfully in 178 msec.

✓ Query returned successfully in 178 msec. ✕

Total rows: Query complete 00:00:00.178 CRLF Ln 137, Col 1

3.- Script de “*poblamiento*” de la Bases de Datos (INSERT)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Dashboard × red_apicola/postgr... × public.apiarios/red... × red_apicola/postgres@PostgreSQL 18* ×

red_apicola/postgres@PostgreSQL 18

📁

📄

✎

🔍

No limit

🛑

▶

⏏

📄

📊

🔄

🔧

?

Query

Query History

144

(1, 2),

145

(2, 2),

146

(3, 1),

147

(4, 2);

148

149

-- Productos - Lotes (M:N)

150

INSERT INTO productos_lotes (id_producto, id_lote) VALUES

151

(1, 1),

152

(1, 3),

153

(2, 2),

154

(3, 1),

155

(4, 4),

156

(5, 2);

157

Execute script

F5

Data Output

Messages

Notifications

INSERT 0 6

Query returned successfully in 60 msec.

Total rows:

Query complete 00:00:00.060

✓ Query returned successfully in 60 msec.

CRLF

Ln 15

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

4.-Script de *actualización* de la información especificada (UPDATE)

The screenshot shows a PostgreSQL IDE interface with a script editor and a data output window. The script contains three UPDATE statements for markets, three UPDATE statements for products, and two SELECT statements for verification. The data output window displays a table with 5 rows of product information.

```
-- Tres actualizaciones de dirección de mercados
UPDATE mercados SET direccion = 'Cra 45 # 30-50 Local 12' WHERE municipio = 'Medellín';
UPDATE mercados SET direccion = 'Calle 50 # 51-30 Plaza Central' WHERE municipio = 'Guarne';
UPDATE mercados SET direccion = 'Cra 31 # 30-15 Galería Municipal' WHERE municipio = 'El Carmen de Viboral';

-- Tres actualizaciones de precio de productos
UPDATE productos SET precio = 19500.00 WHERE nombre = 'Miel de abejas natural 500g';
UPDATE productos SET precio = 16000.00 WHERE nombre = 'Propóleo en gotas 30ml';
UPDATE productos SET precio = 37000.00 WHERE nombre = 'Jalea real 50g';

-- Verificación
SELECT id_mercado, nombre, direccion, municipio FROM mercados ORDER BY municipio;
SELECT id_producto, nombre, precio FROM productos ORDER BY id_producto;
```

	id_producto [PK] integer	nombre character varying (100)	precio numeric (10,2)
1	1	Miel de abejas natural 50...	19500.00
2	2	Propóleo en gotas 30ml	16000.00
3	3	Cera de abeja 250g	12000.00
4	4	Polen fresco 200g	22000.00
5	5	Jalea real 50g	37000.00

Showing rows: 1 to 5 Page No: 1 of 1

Successfully run. Total query runtime: 75 msec. 5 rows affected.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

5.-Script de *eliminación* de la información especificada (DELETE**)**

The screenshot shows a PostgreSQL query editor interface. The top bar displays the connection name 'red_apicola/postgres@PostgreSQL 18'. Below the toolbar, the 'Query' tab is active, showing a SQL script with the following content:

```
1 -- DELETE - Sistema Apícola - Equipo 02
2
3 -- Escenario 1: producto ingresado por error, se elimina
4 INSERT INTO productos (nombre, precio) VALUES ('Polen en cápsula - PRUEBA', 8500.00);
5 DELETE FROM productos WHERE nombre = 'Polen en cápsula - PRUEBA';
6
7 -- Escenario 2: apicultor registrado por error, se elimina
8 INSERT INTO usuarios (nombre, correo, id_rol) VALUES ('Carlos Prueba', 'carlos.prueba@apicola.com', 1);
9 DELETE FROM usuarios WHERE correo = 'carlos.prueba@apicola.com';
10
11 -- Eliminar usuario y sus pedidos (respetar FK)
12 DELETE FROM detalle_pedido WHERE id_pedido IN (SELECT id_pedido FROM pedidos WHERE id_usuario = 5);
13 DELETE FROM pedidos WHERE id_usuario = 5;
14 DELETE FROM usuarios WHERE id_usuario = 5;
```

Below the query editor, the 'Data Output' tab is active, showing the result of the query:

```
DELETE 1
Query returned successfully in 53 msec.
```

A green status bar at the bottom right indicates: '✓ Query returned successfully in 53 msec. ✕'. The bottom status bar shows 'Total rows: Query complete 00:00:00.053' and 'CRLF Ln 4, Col 1'.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

6.- Script de consultas de *listados* con la información especificada (SELECT).

The screenshot shows a PostgreSQL query editor interface. The top part displays a SQL query with several JOINs and a SELECT statement. The bottom part shows the results of the query in a table format.

Query:

```
27 INNER JOIN apiarios a ON a.id_apiario = u.id_apiario
28 INNER JOIN colmenas c ON c.id_colmena = a.id_apiario
29 INNER JOIN sensores_iot s ON s.id_colmena = c.id_colmena
30 ORDER BY u.nombre ASC, a.ubicacion ASC;
31
32 -- #5 Pedidos con usuario, producto y subtotal (3 JOIN)
33 SELECT p.id_pedido, p.fecha, u.nombre AS usuario,
34        pr.nombre AS producto, dp.cantidad,
35        pr.precio, (dp.cantidad * pr.precio) AS subtotal
36 FROM pedidos p
37 INNER JOIN usuarios u ON u.id_usuario = p.id_usuario
38 INNER JOIN detalle_pedido dp ON dp.id_pedido = p.id_pedido
39 INNER JOIN productos pr ON pr.id_producto = dp.id_producto
40 ORDER BY p.fecha DESC;
41
```

Data Output:

id_pedido	fecha	usuario	producto	cantidad	precio	subtotal
1	2026-05-...	Sofia Castillo	Jalea real 50g	2	37000.00	74000.00
2	2026-05-...	Sofia Castillo	Miel de abejas natural 50...	6	19500.00	117000.00
3	2026-05-...	Jorge Herrera	Cera de abeja 250g	4	12000.00	48000.00
4	2026-05-...	Jorge Herrera	Polen fresco 200g	1	22000.00	22000.00
5	2026-05-...	Ana Martínez	Miel de abejas natural 50...	2	19500.00	39000.00
6	2026-05-...	Ana Martínez	Propóleo en gotas 30ml	3	16000.00	48000.00
7	2026-04-...	Sofia Castillo	Miel de abejas natural 50...	5	19500.00	97500.00
8	2026-04-...	Sofia Castillo	Jalea real 50g	1	37000.00	37000.00
9	2026-04-...	Jorge Herrera	Polen fresco 200g	2	22000.00	44000.00

Total rows: 12 Query complete 00:00:00.069

Successfully run. Total query runtime: 69 msec. 12 rows affected.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

7.- Script de consultas de *grupos* con la información especificada (*GROUP BY* y *HAVING*)

The screenshot shows a PostgreSQL query editor interface. The top bar displays the database connection: `red_apicola/postgres@PostgreSQL 18*`. The main editor area contains a SQL script with the following content:

```
27 INNER JOIN apiarios a ON a.id_apiario = u.id_usuario
28 INNER JOIN colmenas c ON c.id_colmena = a.id_apiario
29 INNER JOIN sensores_iot s ON s.id_colmena = c.id_colmena
30 ORDER BY u.nombre ASC, a.ubicacion ASC;
31
32 -- #5 Pedidos con usuario, producto y subtotal (3 JOIN)
33 SELECT p.id_pedido, p.fecha, u.nombre AS usuario,
34        pr.nombre AS producto, dp.cantidad,
35        pr.precio, (dp.cantidad * pr.precio) AS subtotal
36 FROM pedidos p
37 INNER JOIN usuarios u ON u.id_usuario = p.id_usuario
38 INNER JOIN detalle_pedido dp ON dp.id_pedido = p.id_pedido
39 INNER JOIN productos pr ON pr.id_producto = dp.id_producto
40 ORDER BY p.fecha DESC;
41
```

Below the query editor, the "Data Output" tab is active, showing the results of the query. The results are displayed in a table with the following columns: `id_pedido` (integer), `fecha` (date), `usuario` (character varying (100)), `producto` (character varying (100)), `cantidad` (integer), `precio` (numeric (10,2)), and `subtotal` (numeric). The table contains 12 rows of data.

id_pedido	fecha	usuario	producto	cantidad	precio	subtotal
1	2026-05-...	Sofia Castillo	Jalea real 50g	2	37000.00	74000.00
2	2026-05-...	Sofia Castillo	Miel de abejas natural 50...	6	19500.00	117000.00
3	2026-05-...	Jorge Herrera	Cera de abeja 250g	4	12000.00	48000.00
4	2026-05-...	Jorge Herrera	Polen fresco 200g	1	22000.00	22000.00
5	2026-05-...	Ana Martínez	Miel de abejas natural 50...	2	19500.00	39000.00
6	2026-05-...	Ana Martínez	Propóleo en gotas 30ml	3	16000.00	48000.00
7	2026-04-...	Sofia Castillo	Miel de abejas natural 50...	5	19500.00	97500.00
8	2026-04-...	Sofia Castillo	Jalea real 50g	1	37000.00	37000.00
9	2026-04-...	Jorge Herrera	Polen fresco 200g	2	22000.00	44000.00

At the bottom of the interface, a green status bar indicates: "Successfully run. Total query runtime: 78 msec. 12 rows affected." The bottom right corner shows "CRLF" and "Ln 33, Col 1".

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

8.- Script de consultas con *vistas* con la información especificada (*VIEW*)

The screenshot shows a PostgreSQL IDE interface with the following components:

- Query Editor:** Contains a SQL script with 22 lines. The script selects user information and aggregates order data. A tooltip "Execute script" with a green "F5" icon is visible over the script.
- Query:**

```
8 SELECT u.id_usuario, u.nombre AS us_nombre, r.nombre AS rol_nombre,
9 COUNT(DISTINCT p.id_pedido) AS total_pedidos,
10 SUM(dp.cantidad) AS total_unidades,
11 SUM(dp.cantidad * pr.precio) AS total_cop,
12 AVG(pr.precio) AS precio_promedio,
13 MAX(pr.precio) AS precio_max,
14 MIN(pr.precio) AS precio_min
15 FROM usuarios u
16 INNER JOIN roles r ON r.id_rol = u.id_rol
17 INNER JOIN pedidos p ON p.id_usuario = u.id_usuario
18 INNER JOIN detalle_pedido dp ON dp.id_pedido = p.id_pedido
19 INNER JOIN productos pr ON pr.id_producto = dp.id_producto
20 WHERE p.fecha >= '2024-01-01'
21 GROUP BY u.id_usuario, u.nombre, r.nombre
22 HAVING SUM(dp.cantidad * pr.precio) > 10000
```
- Data Output / Messages:** Shows two notices: "view 'vista_resumen_pedidos_por_usuario' does not exist, skipping" and "la vista «vista_sensores_activos_por_apiario» no existe, omitiendo CREATE VIEW". Below these, it states "Query returned successfully in 65 msec."
- Status Bar:** At the bottom, it shows "Total rows: Query complete 00:00:00.065" and a green notification "Query returned successfully in 65 msec." with a close button.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

9.- Script de consultas con *parámetros* con la información especificada (PREPARE**)**

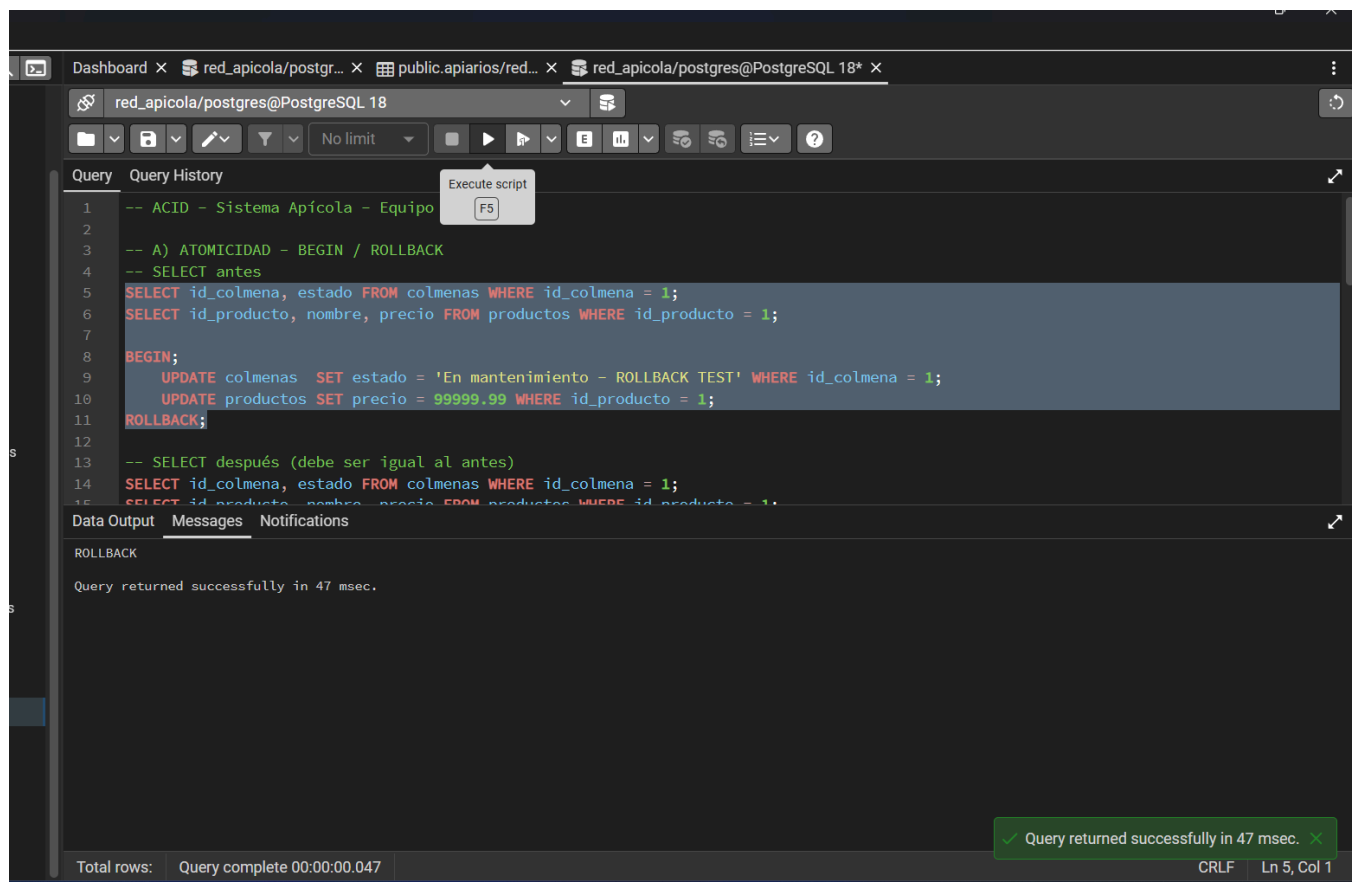
The screenshot shows a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main editor area contains a SQL script with the following content:

```
1 -- PREPARE / Consulta parametrizada Apícola - Equipo 02
2 -- $1 tipo_sensor (TEXT), $2 estado_colmena (TEXT), $3 min_promedio (NUMERIC)
3
4 PREPARE consulta_sensores_por_apiario (TEXT, TEXT, NUMERIC) AS
5     SELECT a.id_apiario, a.ubicacion AS apiario,
6           c.id_colmena, c.estado,
7           s.id_sensor, s.tipo,
8           COUNT(ds.id_datos) AS lecturas,
9           AVG(ds.valor) AS promedio,
10          MAX(ds.valor) AS max,
11          MIN(ds.valor) AS min
12 FROM apiarios a
13 INNER JOIN colmenas c ON c.id_apiario = a.id_apiario
14 INNER JOIN sensores_iot s ON s.id_colmena = c.id_colmena
15 INNER JOIN datos_sensor ds ON ds.id_sensor = s.id_sensor
```

Below the script, the 'Data Output' tab is active, showing the message: 'Query returned successfully in 41 msec.' A green notification bar at the bottom right also displays: '✓ Query returned successfully in 41 msec. ✕'. The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.041' and 'CRLF Ln 27, Col 1'.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

10.- Script de verificación de *propiedades ACID*



The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```
1  -- ACID - Sistema Apícola - Equipo
2
3  -- A) ATOMICIDAD - BEGIN / ROLLBACK
4  -- SELECT antes
5  SELECT id_colmena, estado FROM colmenas WHERE id_colmena = 1;
6  SELECT id_producto, nombre, precio FROM productos WHERE id_producto = 1;
7
8  BEGIN;
9      UPDATE colmenas SET estado = 'En mantenimiento - ROLLBACK TEST' WHERE id_colmena = 1;
10     UPDATE productos SET precio = 99999.99 WHERE id_producto = 1;
11 ROLLBACK;
12
13 -- SELECT después (debe ser igual al antes)
14 SELECT id_colmena, estado FROM colmenas WHERE id_colmena = 1;
15 SELECT id_producto, nombre, precio FROM productos WHERE id_producto = 1;
```

The interface includes a toolbar with an "Execute script" button (F5). Below the query editor, the "Data Output" tab shows the message "ROLLBACK" and "Query returned successfully in 47 msec." A green status bar at the bottom right confirms "Query returned successfully in 47 msec." The status bar also shows "Total rows: Query complete 00:00:00.047" and "CRLF Ln 5, Col 1".

11.- Script de demostración de inserción, actualización y consulta de una dato *JSON*

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Dashboard × red_apicola/postgr... × public.apiarios/red... × red_apicola/postgres@PostgreSQL 18* ×

red_apicola/postgres@PostgreSQL 18

Execute script
FS

```
1 -- JSON/JSONB - Sistema Apícola - B
2
3 ALTER TABLE datos_sensor ADD COLUMN IF NOT EXISTS datos_ambientales JSONB;
4 ALTER TABLE productos ADD COLUMN IF NOT EXISTS características JSONB;
5
6 -- INSERT lecturas IoT
7 INSERT INTO datos_sensor (id_sensor, valor, fecha, datos_ambientales) VALUES
8 (1, 34.5, NOW(),
9 '{"temperatura": 34.5, "humedad": 72, "presion": 1012, "coordenadas": {"latitud": 6.2442, "longitud": -75.5812, "altitud_m": 1495}, "
10 (1, 41.8, NOW() - INTERVAL '1 hour',
11 '{"temperatura": 41.8, "humedad": 65, "presion": 1008, "bateria_pct": 54, "estado": "alerta_temperatura"}'::JSONB),
12 (2, 33.1, NOW() - INTERVAL '2 hours',
13 '{"temperatura": 33.1, "humedad": 79, "presion": 1015, "bateria_pct": 12, "estado": "bateria_baja"}'::JSONB);
14
```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	id_producto [PK] Integer	nombre character varying (100)	certificacion text
1	3	Cera de abeja 250g	[null]
2	4	Polen fresco 200g	[null]
3	1	Miel de abejas natural 50...	[null]
4	2	Propóleo en gotas 30ml	[null]
5	5	Jalea real 50g	[null]
6	8	Propóleo	[null]
7	7	Miel orgánica	organico-premiu...

✓ Successfully run. Total query runtime: 64 msec. 7 rows affected. ✕

Total rows: 7 Query complete 00:00:00.064 CRLF Ln 70, Col 1

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

12.- Análisis de resultados

Durante el desarrollo de la base de datos se ejecutaron diferentes consultas con el propósito de validar la información almacenada y comprobar el correcto funcionamiento de las relaciones entre las tablas. Los resultados obtenidos permitieron verificar que los datos fueron registrados de manera consistente y que las restricciones definidas en el modelo físico garantizan la integridad de la información.

Las consultas de listado permitieron visualizar información relevante sobre los usuarios, apiarios, colmenas, sensores y pedidos registrados en el sistema. Gracias al uso de JOINS fue posible relacionar múltiples tablas y obtener resultados más completos, facilitando la comprensión de cómo interactúan los diferentes elementos del sistema apícola. Estas consultas demostraron que las claves primarias y foráneas fueron implementadas correctamente, ya que los datos relacionados se mostraron de manera precisa y sin inconsistencias.

Por otra parte, las consultas de agrupamiento permitieron generar reportes estadísticos útiles para el análisis de la información. Se identificó la cantidad de apicultores asociados a cada apiario, los productos con mayor número de ventas, los sensores instalados por apiario y los usuarios con mayor actividad de compra. Estos resultados evidencian cómo las funciones de agregación pueden utilizarse para apoyar la toma de decisiones y obtener indicadores relevantes del negocio.

Las vistas creadas facilitaron el acceso a información frecuentemente consultada, reduciendo la complejidad de las consultas y mejorando la organización de los datos. Además, las pruebas realizadas con operaciones de eliminación permitieron verificar que las relaciones entre tablas se mantienen de forma adecuada, evitando la generación de registros huérfanos o inconsistentes.

En términos generales, los resultados obtenidos demuestran que la base de datos cumple con los objetivos planteados para el proyecto, proporcionando una estructura sólida para almacenar, consultar y analizar la información del sistema apícola. Las consultas implementadas permiten obtener datos relevantes de forma eficiente, mientras que el diseño de la base de datos garantiza confiabilidad, organización y facilidad de mantenimiento para futuras ampliaciones del sistema.

13.- Reflexiones y conclusiones individuales

ADER ELIECER QUINTANA HERRERA : Dentro del desarrollo de la Tarea 6 me encargué de tres componentes clave de la manipulación de datos en PostgreSQL: la verificación de las propiedades ACID, el manejo de datos en formato JSON/JSONB y la construcción de consultas parametrizadas con PREPARE.

En la parte de propiedades ACID trabajé los cuatro escenarios exigidos. Para la atomicidad implementé una transacción con BEGIN y ROLLBACK sobre dos tablas, comprobando con consultas SELECT antes y después que ningún cambio quedaba guardado al revertir. Para la consistencia diseñé operaciones que violan deliberadamente las restricciones del modelo (clave primaria duplicada, valor único repetido y borrado de un registro padre con hijos asociados), demostrando que el motor las rechaza y protege la integridad. El aislamiento lo planteé como un caso

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

hipotético de dos sesiones concurrentes bajo READ COMMITTED, y la durabilidad la validé con una transacción BEGIN–COMMIT que persiste el cambio de forma permanente.

En el manejo de datos JSON/JSONB trabajé sobre las columnas de lecturas de sensores IoT y de características de productos. Realicé la inserción de documentos JSON, su consulta con los operadores ->> y -> para extraer campos simples y objetos anidados como las coordenadas GPS, y la actualización de valores internos con el operador de fusión ||. Esto me permitió entender por qué JSONB es la mejor opción para escenarios de Big Data e IoT. En las consultas con parámetros construí una sentencia PREPARE con tres parámetros que combina tres JOIN, filtros WHERE, HAVING y funciones de agregación.

Un recurso fundamental durante todo el proceso fue el diagrama Entidad–Relación, que me ayudó a entender cómo se conectaban las tablas y a escribir correctamente los JOIN y las claves foráneas. Mi principal dificultad fue ajustar los scripts al modelo físico real, lo que me enseñó a validar siempre contra la estructura definitiva. Como aprendizaje, logré una comprensión más sólida del lenguaje DML y del valor de las transacciones para garantizar la calidad de la información.

SAMUEL BRAND GAVIRIA : Resumen de lo aprendido

Durante el desarrollo de esta tarea fortalecí mis conocimientos en bases de datos relacionales utilizando PostgreSQL. Aprendí a realizar operaciones fundamentales como la eliminación de registros mediante sentencias DELETE, teniendo en cuenta las restricciones y relaciones entre tablas para mantener la integridad de los datos. También profundicé en la construcción de consultas SELECT utilizando JOINs para relacionar información de múltiples tablas y obtener resultados más completos y útiles.

Además, comprendí la importancia de las consultas de agrupamiento mediante GROUP BY y HAVING para generar estadísticas y reportes que facilitan el análisis de la información almacenada. Otro aspecto importante fue el uso de vistas (VIEW), las cuales permiten simplificar consultas complejas y reutilizar resultados de manera eficiente. En general, esta actividad me permitió entender mejor cómo administrar, consultar y analizar datos dentro de una base de datos real aplicada a un sistema de gestión apícola.

Mi aporte al proyecto

Mi principal aporte al proyecto fue el desarrollo de los scripts de eliminación, consultas de listados, agrupamientos y vistas. En el script DELETE me encargué de eliminar registros de prueba y datos obsoletos de diferentes tablas, verificando que las relaciones entre ellas no generaran errores de integridad referencial.

También desarrollé consultas SELECT para mostrar información relevante del sistema, incluyendo apiarios, usuarios, roles, colmenas, sensores y pedidos, utilizando JOINs para relacionar correctamente los datos. Adicionalmente, elaboré consultas con GROUP BY y HAVING para generar reportes estadísticos, como la cantidad de apicultores por apiario, los productos más vendidos, los sensores registrados y los usuarios con mayor actividad de compra.

Finalmente, participé en la creación de vistas que permiten acceder de forma más rápida y organizada a la información, específicamente una vista de resumen de pedidos por usuario y otra relacionada con los sensores activos por apiario, incluyendo promedios y valores mínimos y máximos registrados.

FRANKLIN ALEJANDRO TORRES FLOREZ Durante el desarrollo de la asignatura Bases de Datos I, se fortalecieron conocimientos fundamentales relacionados con el diseño, construcción y administración de bases de datos relacionales. A través de las diferentes actividades realizadas, fue posible comprender la importancia de una

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

correcta organización de la información y de cómo una estructura bien diseñada permite optimizar procesos, mantener la integridad de los datos y facilitar futuras consultas dentro de un sistema.

Uno de los aprendizajes más significativos fue la creación de tablas y relaciones entre ellas, entendiendo la función de las claves primarias y foráneas para garantizar la conexión lógica entre la información almacenada. Asimismo, se trabajó en la construcción de tablas de unión, permitiendo representar relaciones entre distintas entidades y mejorar la estructura general de la base de datos. Este proceso ayudó a comprender cómo los datos se relacionan entre sí y cómo pueden organizarse de manera eficiente para evitar redundancia e inconsistencias.

Como parte del aporte realizado dentro del desarrollo práctico, participé activamente en la creación de tablas, en la implementación de uniones necesarias para el correcto funcionamiento de la base de datos, en la ejecución de instrucciones UPDATE para modificar y mantener actualizada la información, y en el poblamiento de tablas mediante el ingreso de registros necesarios para simular el funcionamiento real del sistema. Estas actividades permitieron fortalecer habilidades técnicas relacionadas con el manejo de sentencias SQL y la lógica estructural de las bases de datos.

Finalmente, esta experiencia permitió comprender que el diseño y mantenimiento de una base de datos requiere organización, análisis y precisión, ya que pequeños errores pueden afectar el funcionamiento completo del sistema. Además, dejó como enseñanza la importancia del trabajo práctico para reforzar los conceptos teóricos, desarrollando una mejor comprensión sobre la gestión eficiente de la información dentro de entornos tecnológicos.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Rúbrica: Criterios de Evaluación de la Tarea

#	Ítems Tarea	Peso	Cal
1	Inventario de tablas definitivo (corregido, si aplica)	10	
2	Script de creación de la base de datos (CREATE)	30	
3	Script de “ poblamiento ” de las tablas especificadas (INSERT)	80	
4	Script de actualización de la información especificada (UPDATE)	10	
5	Script de eliminación de la información especificada (DELETE)	10	
6	Script de consultas de listados con la información especificada (SELECT).	70	
7	Script de consultas de grupos con la información especificada (SELECT)	80	
8	Script de consultas con vistas con la información especificada (VIEW)	20	
9	Script de consultas con parámetros con la información especificada (PREPARE)	30	
10	Script de verificación de propiedades ACID	40	
11	Script de demostración de inserción, actualización y consulta de una dato JSON	10	
12	Análisis de resultados (en equipo - 300 palabras mínimo)	20	
13	Reflexiones y Conclusiones individuales (300 palabras mínimo)	30	
14	Informe: Uso de Plantillas, Estructura, Contenido y Calidad de presentación	40	
15	Video de Sustentación	150	
16	Repositorio GIT	20	
	NOTA = xx/500 =	Total	650
B	BONO ESPECIAL	RESPALDO PLANO DE LA BASE DE DATOS	100

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO A
Propiedades ACID

Propiedad	Descripción	Ejemplo
ATOMICIDAD (Atomicity)	Una transacción: ✓ se ejecuta completamente ✗ o no se ejecuta en absoluto <i>“O todo o nada”</i>	<pre>BEGIN; UPDATE cuenta SET saldo = saldo - 100 WHERE id = 1; UPDATE cuenta SET saldo = saldo + 100 WHERE id = 2; ROLLBACK;</pre> Resultado: no pasó nada
CONSISTENCIA (Consistency)	La base de datos siempre pasa de un estado válido a otro válido. - Se respetan reglas: - claves primarias - claves foráneas - CHECK - tipos de datos <i>“Datos válidos”</i>	<pre>INSERT INTO paciente (id_paciente) VALUES (NULL);</pre> Resultado: ✗ Falla si hay restricción → mantiene consistencia
AISLAMIENTO (Isolation)	Las transacciones no se “estorban” entre sí. - Cada usuario ve su propia transacción - Evita problemas como: - lecturas sucias - lecturas fantasma <i>“Transacciones independientes”</i>	Ejemplo conceptual - Usuario A actualiza datos - Usuario B no ve cambios hasta que A haga COMMIT
DURABILIDAD (Durability)	“Si se guardó, se queda guardado” <i>“Persistencia”</i> <i>“COMMIT no es solo guardar... es una promesa de que nunca se perderá.”</i>	<pre>BEGIN; INSERT INTO prueba_durabilidad (mensaje) VALUES ('Dato importante - no se debe perder');</pre> <pre>COMMIT;</pre> Resultado: ✓ Los datos sobreviven: - caídas del sistema - reinicios - fallos

“ACID es lo que evita que tu base de datos se convierta en caos.”

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO B

Cuadro Departamento-Municipios de Apiarios

Departamento	Municipio	Total Municipios	Productores	Apiarios	Consumidores
Antioquia	Caucasia	5	2	3	50
	El Bagre		2	3	50
	Zaragoza		2	3	50
	Santa Fe de Antioquia		3	5	50
	Medellín (Santa Elena)		5	10	100
Cundinamarca	Fusagasugá	5	3	5	50
	Girardot		2	5	50
	Ubaté		3	3	50
	Zipaquirá		3	5	50
	La Mesa		1	3	50
Boyacá	Tunja	5	4	10	50
	Duitama		3	3	50
	Sogamoso		2	5	50
	Chiquinquirá		1	3	50
	Paipa		1	3	50
Santander	Bucaramanga	4	4	8	50
	San Gil		1	3	50
	Socorro		2	3	50
	Barbosa		1	3	50
Huila	Neiva	4	4	10	50
	Pitalito		1	3	50
	Garzón		2	3	50
	La Plata		2	3	50
Meta	Villavicencio	3	4	9	50
	Granada		2	3	50
	Acacías		3	3	50
Córdoba	Montería	3	3	8	50
	Planeta Rica		1	3	50
	Sahagún		1	3	50
Sucre	Sincelejo	3	3	9	50
	Corozal		3	6	50
	Sampués		1	3	50
Magdalena	Santa Marta	3	4	9	50
	Ciénaga		3	5	50
	Fundación		3	5	50
Valle del Cauca	Cali	4	5	8	50
	Palmira		3	6	50
	Buga		3	7	50
	Tuluá		4	8	50
	TOTALES	37	100	200	2000

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO C

Cuadro de Productos con unidad de medida y costo aproximado (solo para ejercicio académico)

Código	Producto	Unidad típica	Precio aproximado (COP)
100	 Miel	1 kg	\$25.000
200	 Polen	250 g	\$20.000
300	 Propóleo	30 ml (extracto)	\$35.000
400	 Jalea Real	10–20 g	\$40.000
500	 Cera	1 kg	\$30.000
600	 Apitoxina	1 g	\$300.000

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO D

Especificaciones de Pedidos

1. Se está lanzando la Red Apícola apenas.
2. No todos los productores elaboran los 6 productos. Usted los asignará libremente.
3. Usted registrará 2000 pedidos libremente. Deben haber pedidos en todos los departamentos pero no necesariamente en todos los municipios.
 - Antioquia
 - Cundinamarca
 - Boyacá
 - Santander
 - Huila
 - Meta
 - Córdoba
 - Sucre
 - Magdalena
 - Valle del Cauca
4. Obligatoriamente, deben haber pedidos en los siguientes municipios:
 - Santa Fe de Antioquia
 - Medellín (Santa Elena)
 - Fusagasugá
 - Girardot
 - Tunja
 - Sogamoso
 - Bucaramanga
 - Neiva
 - Villavicencio
 - Monteria
 - Sincelejo
 - Corozal
 - Santa Marta
 - Ciénaga
 - Cali
 - Palmira
5. No necesariamente todos los consumidores harán pedidos
6. No necesariamente todos los productores recibirán pedidos
7. Los pedidos pueden tener uno o más tuplas en el detalle. Usted decide arbitrariamente. Cada tupla de detalle debe incluir el número de pedido, el código de producto, la cantidad y el precio (el precio puede cambiar eventualmente en la tabla productos y se debe conservar la información de venta original)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO E

Ejemplo de Tabla con información No Relacional
Tipos de Datos Semi-Estructurados (JSON/JSONB)

Tabla “apiario”

Campo: *ubicacion_geografica* JSONB

La ubicación no es solo latitud/longitud. En el mundo real incluye:

- Información jerárquica (departamento, municipio)
- Características del terreno
- Tipo de zona (rural/urbana/protegida)
- Metadatos adicionales

Esto hace que sea ideal como estructura semi-estructurada

Ejemplo de JSON

```
{
  "coordenadas": {
    "latitud": 6.25184,
    "longitud": -75.56359
  },
  "altitud_msnm": 1550,
  "region": {
    "pais": "Colombia",
    "departamento": "Antioquia",
    "municipio": "Medellín",
    "vereda": "Santa Elena"
  },
  "tipo_zona": "rural",
  "condiciones_ambientales": {
    "flora_dominante": ["eucalipto", "café"],
    "fuente_agua_cercana": true
  }
}
```

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO F

Ejemplo de inserción y consulta de datos en la Tabla “apiario”

Inserción de información

```
INSERT INTO apiario (nombre, ubicacion_geografica)
VALUES
(
    'Apiario La Montaña',
    '{
        "coordenadas": {
            "latitud": 6.25184,
            "longitud": -75.56359
        },
        "altitud_msnm": 1550,
        "region": {
            "pais": "Colombia",
            "departamento": "Antioquia",
            "municipio": "Medellín",
            "vereda": "Santa Elena"
        },
        "tipo_zona": "rural",
        "condiciones_ambientales": {
            "flora_dominante": ["eucalipto", "café"],
            "fuente_agua_cercana": true
        }
    }'
),
(
    'Apiario El Bosque',
    '{
        "coordenadas": {
            "latitud": 4.7110,
            "longitud": -74.0721
        },
        "altitud_msnm": 2600,
        "region": {
            "pais": "Colombia",
            "departamento": "Cundinamarca",
            "municipio": "Bogotá",
            "vereda": "Usme"
        },
        "tipo_zona": "periurbana",
        "condiciones_ambientales": {
            "flora_dominante": ["pino", "acacia"],
            "fuente_agua_cercana": false
        }
    }'
);
```

Consultas de verificación

```
SELECT * FROM apiario
WHERE ubicacion_geografica->'region'->>'departamento' = 'Antioquia';

SELECT * FROM apiario
WHERE (ubicacion_geografica->>'altitud_msnm')::int > 1500;
```